CAPSTONE · PTY4614 · SECCIÓN 005D

Documento de diseño

Sistema de Punto de Venta e Inventario — Emporio NaturalSur

1. Propósito y alcance **2**
2. Contexto y restricciones de diseño **2**
3. Arquitectura de la solución **2**
a. Vista general 2
b. Componentes y responsabilidades 2
c. Comunicación entre componentes 2
d. Flujo de la funcionalidad principal: registrar una venta 2
e. Operación si conexión y sincronización 2
4. Modelo de datos **2**
a. Tablas por área 2
b. Decisiones del modelo que corrigen los defectos del legado 2
5. Diagramas UML **2**
6. Diseño de la interfaz **2**
7. Decisiones técnicas y su justificación **2**
8. Exención de contenedores: justificación **2**
a. Por qué no aplica 2
b. Mecanismos equivalentes 2
9. Diseño de la migración **2**
10. Seguridad y datos personales **2**
11. Temas abiertos de diseño **2**

Dato	Valor
Versión	1.1 · 19-10-2026
Base	Decisiones de Fernando Silva en los ADR 0001 a 0003 (24-08-2026) y su síntesis del 09-10-2026 (rama docs/nfr-y-diseno); modelo de datos de 20 tablas (commit cd87850); diagramas UML del 08-10-2026
Responde a	Instructivo Capstone: documento de diseño del marco ágil (10 a 15 páginas) y obligatorios transversales de arquitectura, modelo de datos y diagramas UML

Dato	Valor
Documentos relacionados	Product Vision v2.2 · Product Backlog v4.4 · Requisitos no funcionales v1.1 · Plan de pruebas v3.1 · Definition of Done · Diagramas UML

1. Propósito y alcance

Este documento describe cómo está construido el sistema y por qué. Reúne la arquitectura, los componentes y su comunicación, el modelo de datos, los diagramas UML, el diseño de la interfaz y las decisiones técnicas con sus alternativas descartadas. Incluye además la sección justificatoria de la exención de contenedores comprometida ante la docente guía.

El qué del sistema está en el Product Backlog (49 historias de usuario) y las condiciones que debe cumplir, en el documento de requisitos no funcionales. Este documento no los repite: los referencia por su identificador.

2. Contexto y restricciones de diseño

El sistema reemplaza a Abarrotes Punto de Venta 2.12, un software de 2010 sin soporte que el local heredó con el negocio. El diagnóstico, hecho sobre una copia de su base de datos, mostró que el margen declarado es de 53,4% cuando el real es 34,0%, que el cierre de caja no suma las ventas a crédito en 144 de 145 jornadas, y que no existe control de vencimientos. Esos hallazgos son requisitos de diseño, no solo de funcionalidad.

Restricción	Consecuencia en el diseño
Un solo computador con Windows 8.1, sin soporte desde enero de 2023, que no se reemplazará	Descarta las tecnologías que ya no corren en Windows 8.1 y obliga a Python 3.11 (sección 8, ADR 0001)
Las propietarias exigieron poder vender sin internet	La base local es la fuente de verdad y la nube se alimenta por una cola (sección 3.5, ADR 0002)
No hay personal técnico en el local ni presupuesto para infraestructura	Sin servidores propios: la base remota es un servicio gestionado (Supabase) y el sistema se instala como un ejecutable
Sin UPS	Base local en modo WAL, que no se corrompe ante un corte (RNF-DIS-03)
Periféricos existentes: lector USB, impresora de 58 mm, cajón, balanza sin puerto	Se reutilizan; la balanza no se integra y el peso se digita
Datos del legado con defectos medidos	El modelo nuevo impide reproducirlos por construcción (sección 4.2)

3. Arquitectura de la solución

a. Vista general

La solución tiene tres partes: Una aplicación de escritorio que corre en el computador del local y guarda todo en una base SQLite local; una base remota en Supabase que recibe una copia de cada operación cuando hay conexión; y un proceso de migración, que se ejecuta una sola vez, para traer el catálogo y las cuentas de clientes desde el sistema legado.

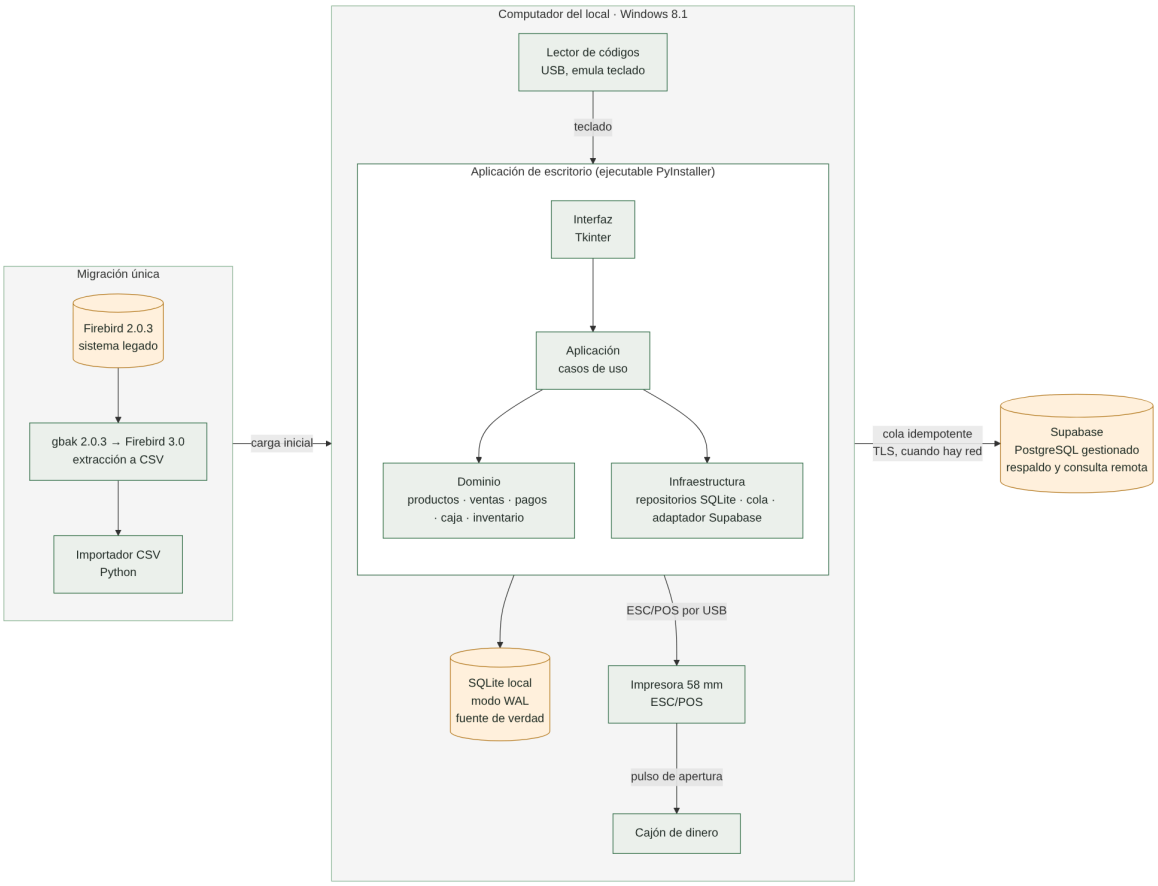


Figura 1. Vista general de la arquitectura

b. Componentes y responsabilidades

La aplicación se organiza en cuatro capas. Cada capa solo conoce a la que está por debajo, y el dominio no depende de ninguna tecnología: las reglas del negocio se prueban sin base de datos ni interfaz.

Capa	Módulos (src/pos/)	Responsabilidad
Interfaz	interfaz/app_tkinter.py	Pantallas y atajos de teclado. Recibe el código del lector como si fuera teclado. No contiene reglas de negocio

Capa	Módulos (src/pos/)	Responsabilidad
Aplicación	aplicacion/casos_uso/: registrar producto, escanear código, ajustar stock, registrar venta, abrir y cerrar turno de caja, registrar movimiento de caja, sincronizar pendientes · aplicacion/puertos.py	Orquesta cada operación de principio a fin y define, como puertos, lo que necesita de la infraestructura
Dominio	dominio/: productos, ventas, pagos, caja, inventario, código de barras, valores (Dinero), errores	Reglas del negocio: margen y markup, redondeo del efectivo, arqueo, existencias, lectura de códigos con peso
Infraestructura	infraestructura/sqlite/: conexión, esquema, repositorios, cola de sincronización · infraestructura/supabase/: adaptador y traductor · infraestructura/config.py	Implementa los puertos: persistencia local, publicación a Supabase y lectura de la configuración desde variables de entorno

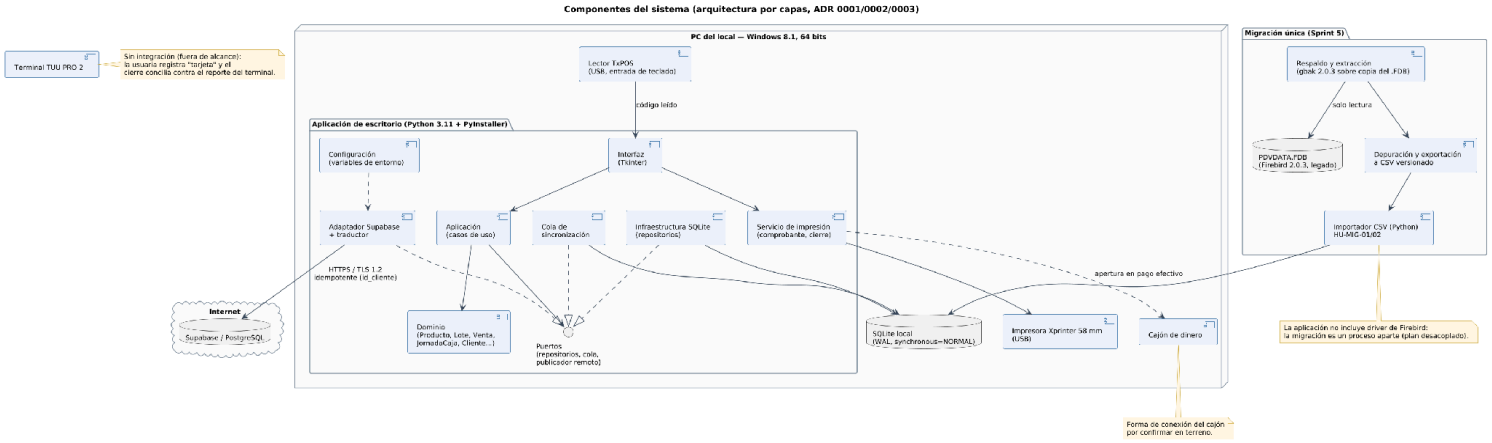


Figura 2. Diagrama UML de componentes

c. Comunicación entre componentes

Desde → hacia	Mecanismo	Modo	Observación
Lector → interfaz	USB en modo teclado: escribe el código y Enter	Síncrono	No requiere controlador
Interfaz → aplicación	Llamada directa a un caso de uso	Síncrono	En el mismo proceso
Aplicación → dominio	Llamada directa a entidades y servicios	Síncrono	Sin dependencias externas
Aplicación → SQLite	Repositorios a través de puertos	Síncrono, transaccional	Cada venta se guarda completa o no se guarda
Cola → Supabase	Protocolo de PostgreSQL con psycopg2, cifrado con TLS (sslmode=require)	Asíncrono, con reintentos	Idempotente por identificador de operación

Desde → hacia	Mecanismo	Modo	Observación
Aplicación → impresora	Comandos ESC/POS por USB	Síncrono	Pendiente de implementar (Sprint 5)
Impresora → cajón	Pulso de apertura por el puerto del cajón	Síncrono	Al cobrar en efectivo
Importador → SQLite	Lectura de CSV y carga	Una sola vez	Fuera de la aplicación (sección 9)

d. Flujo de la funcionalidad principal: registrar una venta

- La vendedora lee el código con el lector o lo busca por nombre. Si el código trae peso embebido, el dominio calcula la cantidad; si es granel sin código de balanza, la interfaz pide el peso
- El caso de uso agrega la línea a la venta. Leer dos veces el mismo producto suma cantidad en vez de duplicar la línea.
- Al cobrar, el dominio valida que los pagos cubran el total. Si hay efectivo, redondea a la decena solo esa parte y calcula el vuelto.
- El caso de uso guarda en una sola transacción la venta, sus pagos y el descuento de existencias. Si falta stock, no se guarda nada.
- La operación se encola para Supabase y la venta queda cerrada sin esperar la red.

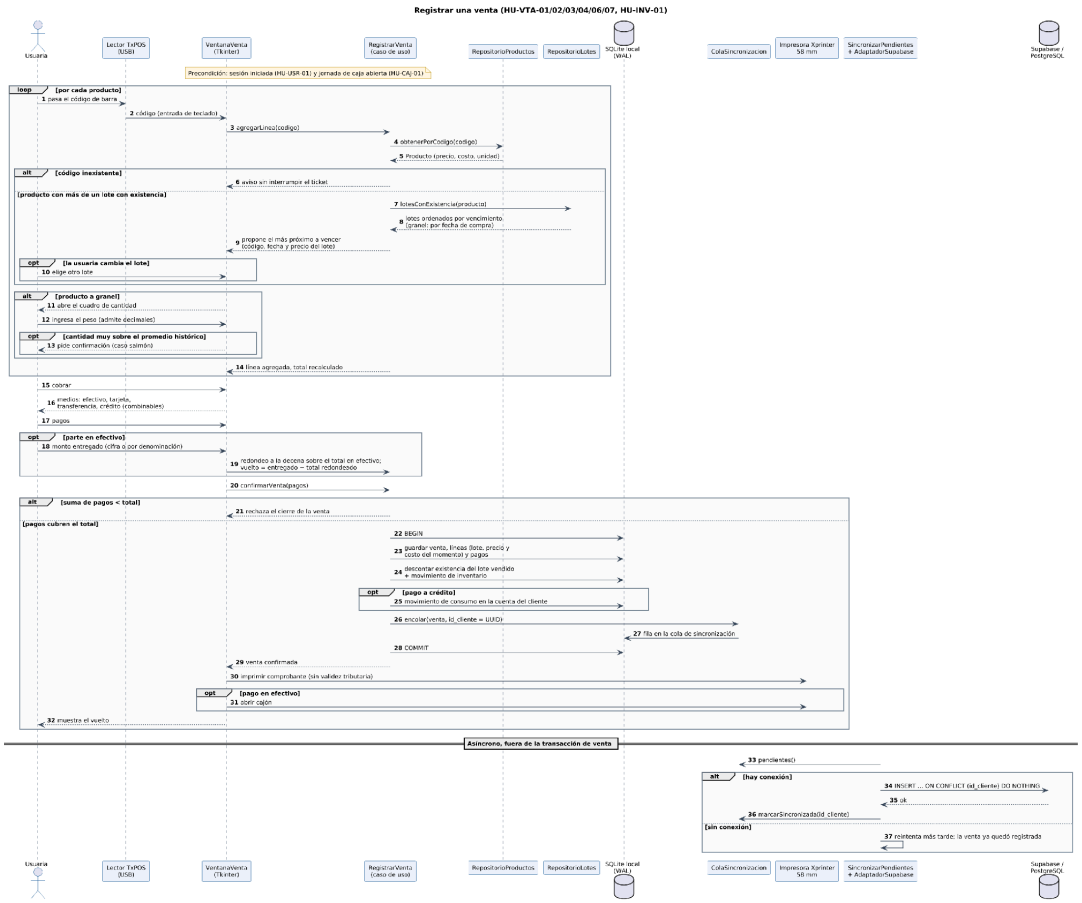


Figura 3. Diagrama UML de secuencia de la venta

e. Operación sin conexión y sincronización

Es la decisión de arquitectura más importante (ADR 0002). La aplicación lee y escribe siempre contra SQLite; Supabase nunca está en el camino de una venta. Cada operación relevante se registra en una cola local con un identificador único generado en el cliente. Un proceso publica la cola en orden cuando hay conexión, y el destino ignora un identificador que ya recibió de modo que un reintento no duplica nada.

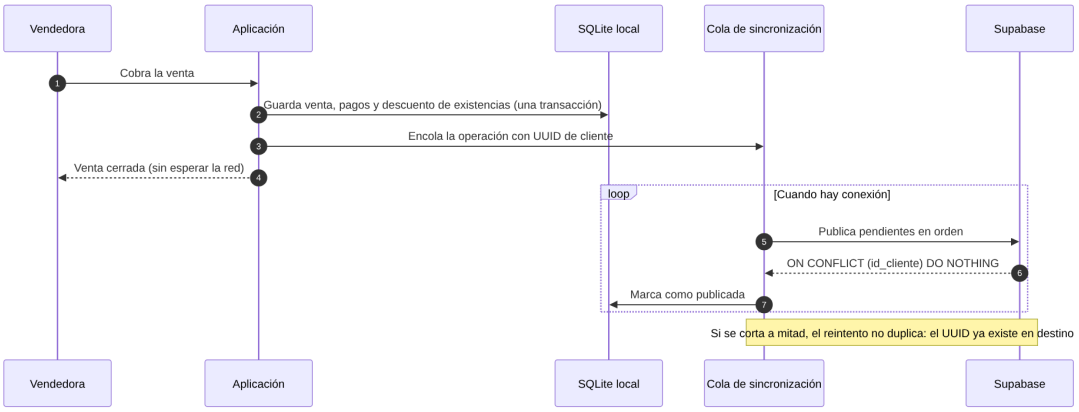


Figura 4. Secuencia de la sincronización

Verificado: La prueba automatizada test_corte_a_mitad_no_pierde_ni_duplica simula un corte en medio de la publicación y comprueba que no se pierde ni se repite ninguna operación.

Pendiente: El indicador visible de operaciones sin sincronizar y la numeración del comprobante sin saltos tras sincronizar (RNF-DIS-05 y RNF-DIS-06).

4. Modelo de datos

El modelo tiene **20 tablas**, definidas en database/migraciones/002_modelo_completo.sql y desplegadas en Supabase. La base local SQLite usa el mismo modelo para las tablas que la aplicación ya ocupa.

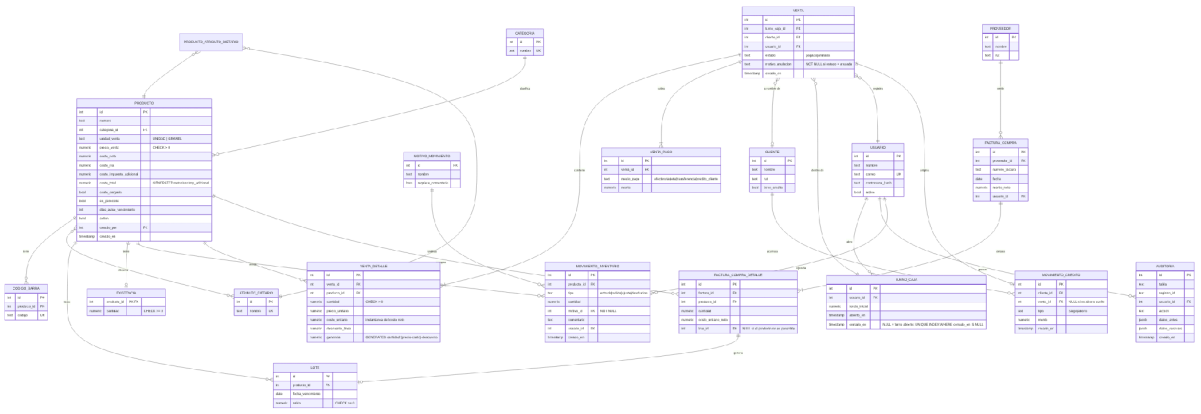


Figura 5. Modelo entidad-relación

a. Tablas por área

Área	Tablas
Catálogo	categoria, producto, codigo_barra, atributo_dietario, producto_atributo_dietario
Inventario	lote, existencia, movimiento_inventario, motivo_movimiento
Caja y ventas	turno_caja, venta, venta_detalle, venta_pago
Clientes con crédito	cliente, movimiento_credito
Compras	proveedor, factura_compra, factura_compra_detalle
Seguridad	usuario, auditoria

b. Decisiones del modelo que corrigen los defectos del legado

Decisión	Defecto que impide
La ganancia de la línea es una columna calculada a partir de cantidad, precio, costo y descuento	El legado guarda la ganancia de la unidad completa sin multiplicar por la cantidad en el granel, y llega a declarar 148,3 % de ganancia
La línea de venta guarda el costo vigente al vender	Hoy el costo histórico no existe y todo margen retrospectivo usa el costo actual
El costo se guarda separado en neto, IVA e impuesto adicional, con el total calculado	El legado mezcla los tres y no permite saber la ganancia neta
costo_cargado distingue «sin costo» de «costo cero»	358 fichas con costo cero que el legado muestra con 100 % de margen
La fecha de vencimiento vive en el lote, no en el producto	Un mismo producto convive con lotes que vencen en fechas distintas
La categoría es una tabla referenciada	El 80,3 % del catálogo legado no tiene departamento; depurar exige renombrar y fusionar categorías
El saldo del cliente se deriva de sus movimientos	El legado guarda un acumulador que se sobrescribe
Un índice único impide dos turnos de caja abiertos	Ventas asignadas a jornadas equivocadas
La anulación exige motivo y conserva la venta	Ventas eliminadas sin rastro

Restricción pendiente de retirar

El script conserva CHECK (NOT costo_cargado OR precio_venta >= costo_total), que impide guardar un precio menor que el costo. Contradice HU-PRD-09, que pide advertir y no bloquear; rechazaría al migrar las cinco fichas que hoy son liquidaciones legítimas; y bloquearía HU-INV-06 si el descuento por vencimiento se aplica bajando el precio del producto. La regla solo existe en Supabase, no en la base local. Según el backlog, el control pasa al formulario como advertencia. Debe retirarse antes de llevar el esquema a producción.

5. Diagramas UML

El instructivo exige cuatro: casos de uso, clases, secuencia de la funcionalidad principal y componentes. Los cuatro están completos en el documento Diagramas UML (Fase 2/Evidencias de Proyecto) y sus fuentes en docs/07-diagramas-uml. Aquí se incluyen los casos de uso y clases; la secuencia y los componentes ya aparecen en las figuras 2 y 3.

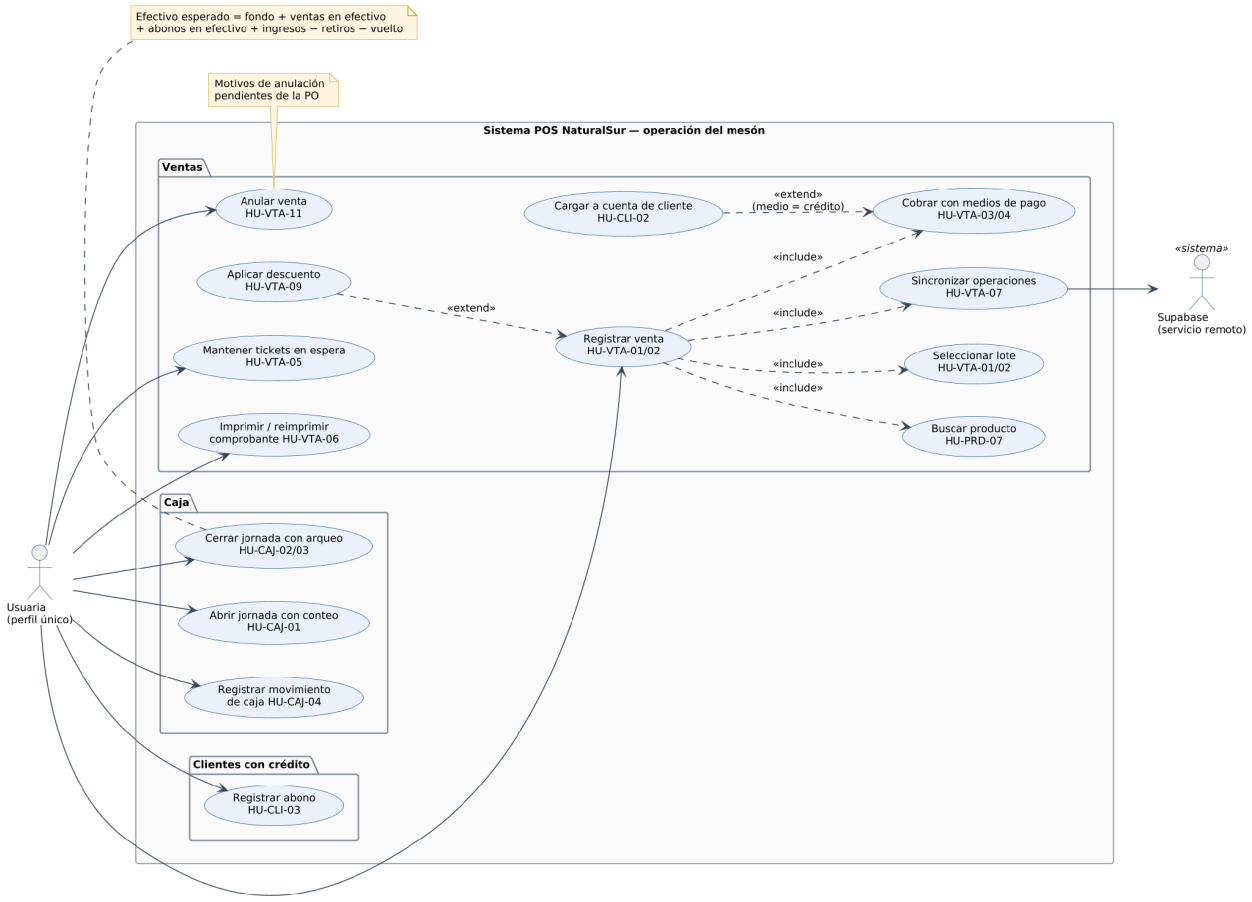
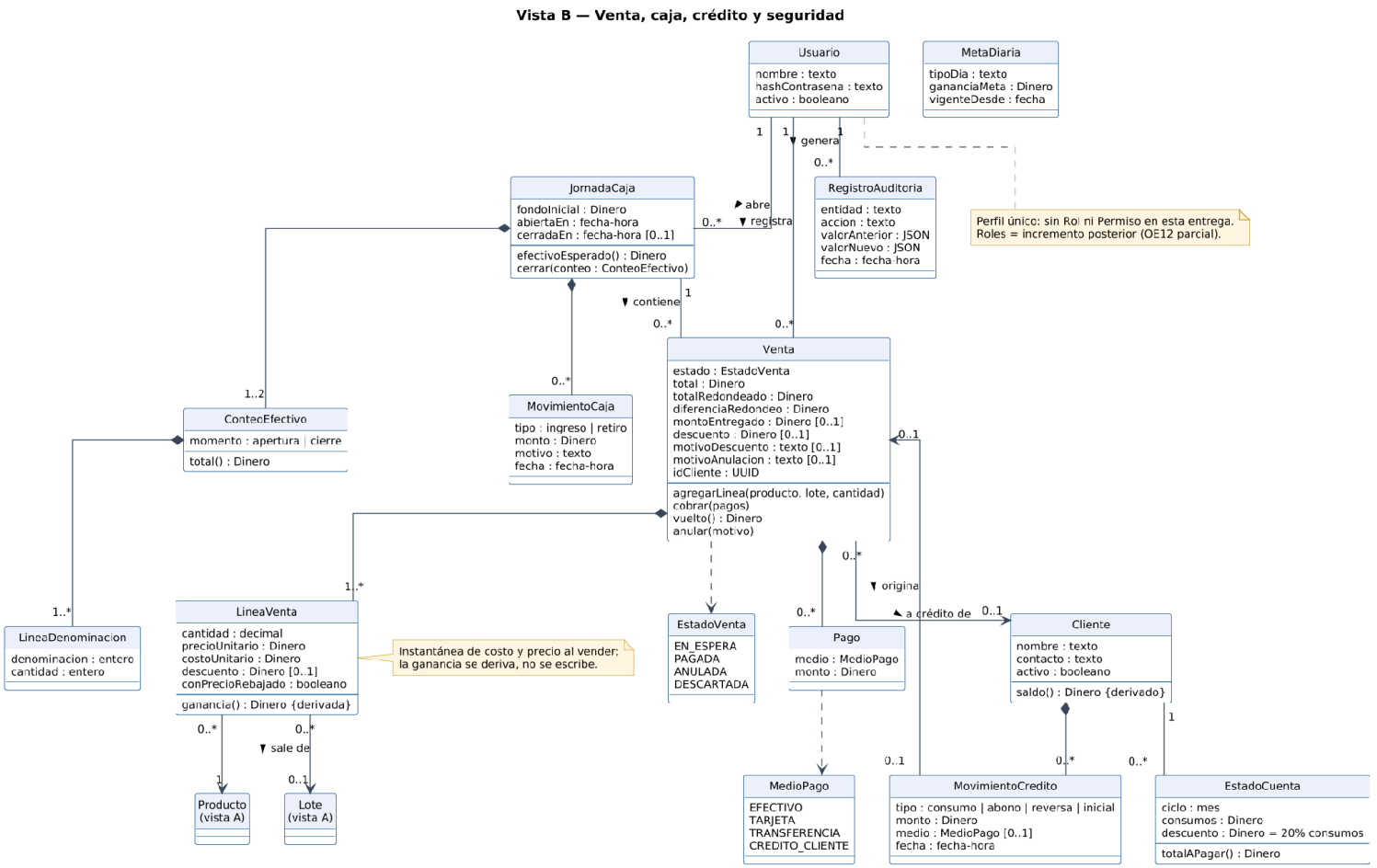


Figura 6. Casos de uso del mesón



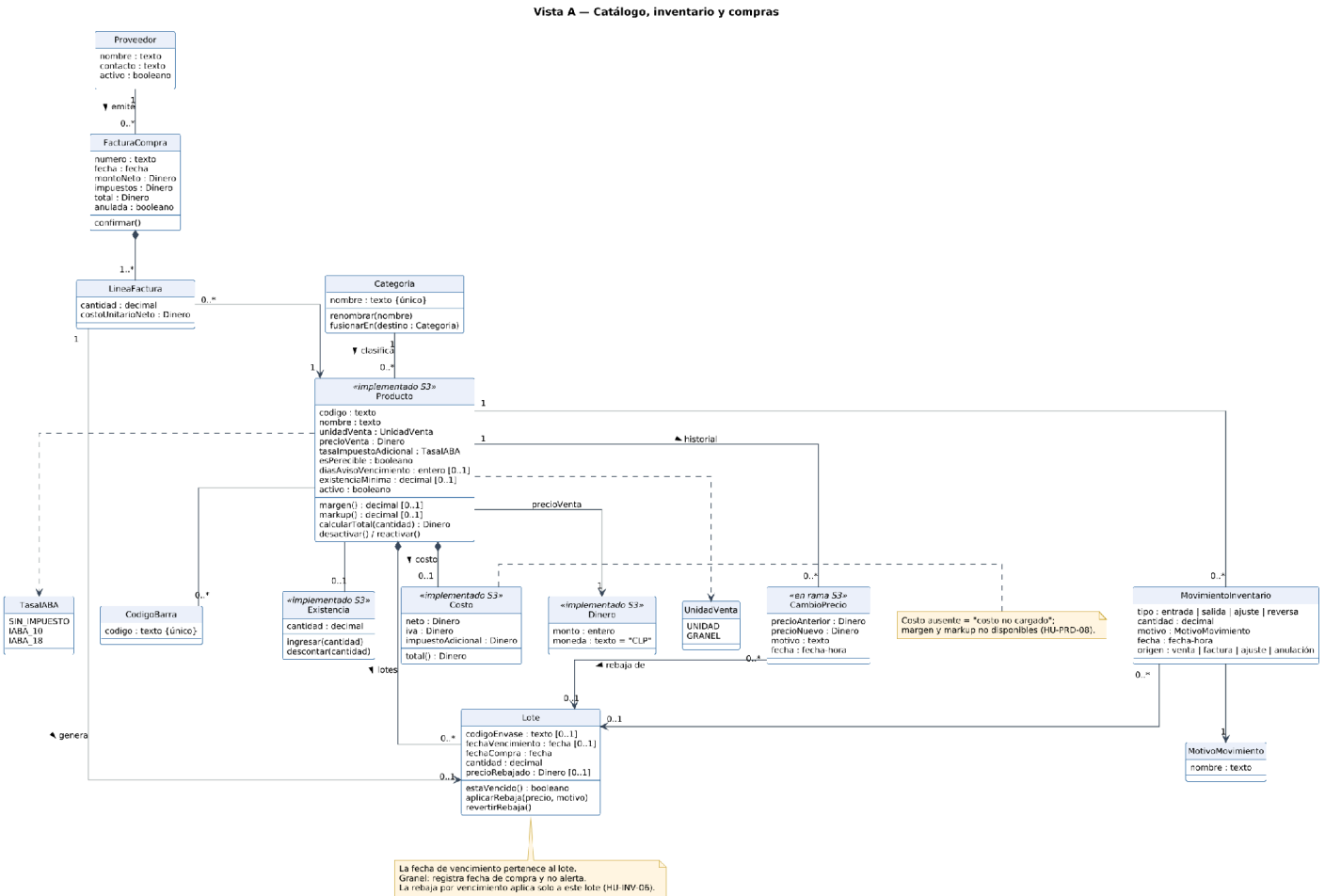


Figura 8. Diagrama de clases del catálogo

6. Diseño de la interfaz

Antes de escribir código se construyó un prototipo navegable (docs/08-diseno/prototipo-naturalsur-pantalla-completa.html) para validar el diseño cuando corregirlo todavía no tenía costo. El criterio principal fue conservar la mecánica del sistema actual para reducir el aprendizaje de las usuarias (RNF-USA-01)

- Módulos arriba, con teclas de función (F1 Operación, F2 Catálogo, F3 Inventario…), y botonera de acciones con su atajo.
- El cursor queda en el campo de código al abrir la venta: el lector escribe y agrega con Enter.
- Varios tickets abiertos a la vez, que no descuentan existencias hasta cobrarse.
- Total grande abajo a la derecha, y estado de sincronización y de la jornada en la barra inferior.
- Margen y markup siempre calculados y en solo lectura.

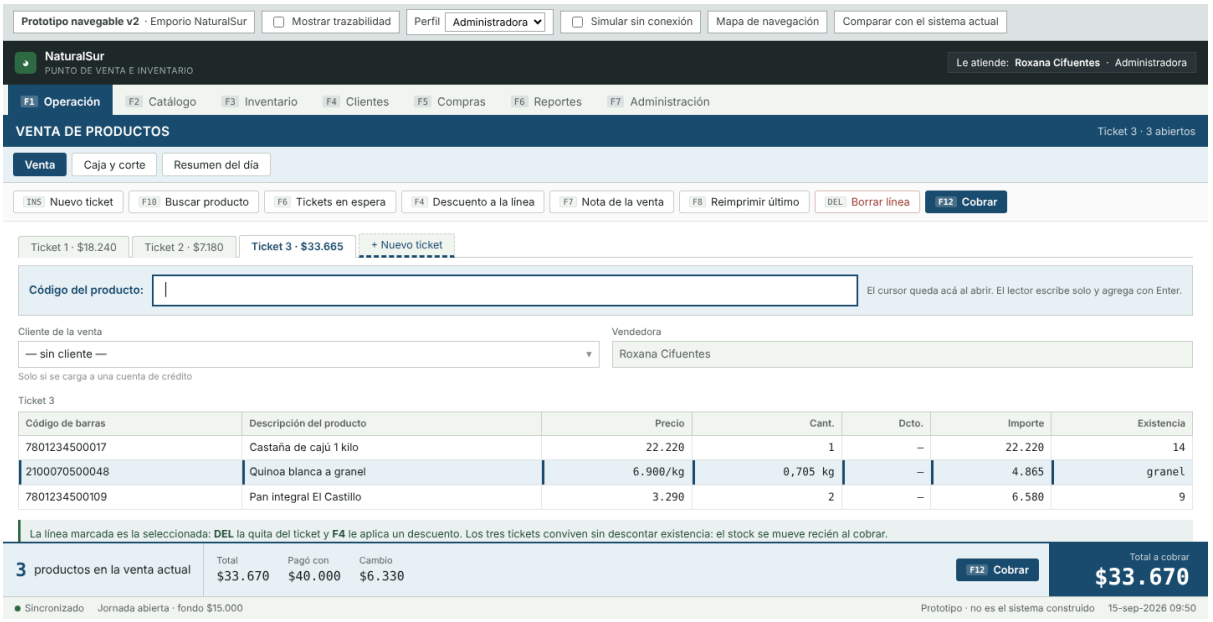


Figura 9. Pantalla de venta del prototipo navegable (datos de muestra)

La aplicación construida a la fecha implementa la pantalla de productos en Tkinter, mostrada en la demostración del 09-10-2026. La pantalla de venta se construye en el Sprint 5 sobre el dominio de ventas y caja que ya está probado.

7. Decisiones técnicas y su justificación

Decisión	Alternativas descartadas	Justificación
Python 3.11 de 64 bits, Tkinter y PyInstaller (ADR 0001)	Python 3.12 o superior; PyQt5; Electron; Tauri; .NET 7 u 8	Es la última versión de Python con soporte oficial para Windows 8.1: Python 3.12 lo dejó porque su soporte extendido terminó el 10-01-2023. Tkinter viene incluido y no requiere runtime adicional. Electron 23+, Tauri y .NET moderno no corren en Windows 8.1
Base local SQLite como fuente de verdad, con cola hacia Supabase (ADR 0002)	Escribir directo a Supabase	Vender sin internet es requisito de las propietarias; depender de la red sería una regresión frente al sistema actual
Supabase como base remota	Servidor propio en AWS (EC2 o ECS)	Servicio gestionado: no hay quien mantenga ni pague un servidor después de diciembre. AWS queda como proyección futura
psycopg2-binary para la conexión	Cliente sobre el cifrado de Windows	Trae su propia biblioteca de cifrado, así la conexión TLS no depende de un Windows sin parches
Empaquetado sin contenedores (ADR 0003)	Docker	Ver sección 8
Migración como evento único	Sincronización permanente entre ambos sistemas	Mantener inventarios sincronizados en dos sistemas es complejo y riesgoso; se ensaya en máquina virtual y se conserva el respaldo como reversa
Lectura de códigos con peso embebido (prefijos 20 a 29)	Integrar la balanza	La balanza del local no tiene puerto; leer el código impreso no depende del modelo de balanza
La tarjeta es un solo medio de pago	Débito y crédito por separado	El terminal de pago no distingue al cobrar; el desglose lo entrega su propio cierre
Redondeo a la decena solo en efectivo, hacia abajo cuando termina en 5	Redondear el vuelto o todo pago	Regla levantada en entrevista; tarjeta y transferencia conservan el monto exacto
Perfil único con auditoría	Roles diferenciados en esta entrega	Acordado con la Product Owner mientras el personal sea familiar; la auditoría se conserva para incorporar roles sin rehacerla
Pruebas con pytest y análisis estático con ruff	—	Estándar de la comunidad, sin costo y ejecutable en el mismo entorno del equipo

8. Exención de contenedores: justificación

El instructivo pide Docker. La docente guía, Karla Roco, autorizó la exención el 21-08-2026 y la ratificó en clase el 28-08-2026, con la condición de documentarla y cumplir mecanismos equivalentes de portabilidad. Esta sección es esa justificación.

a. Por qué no aplica

- El sistema no ejecuta servicios propios. La aplicación es un programa de escritorio y la base remota la provee un tercero como servicio gestionado. No hay nada que contenerizar.
- Docker Desktop no corre en Windows 8.1: requiere Windows 10 u 11. El equipo de destino real no podría ejecutarlo.
- Agregaría una capa que nadie en el local puede mantener.

b. Mecanismos equivalentes

Mecanismo	Dónde está	Estado al 16-10
Ejecutable autocontenido	build/pos.spec (PyInstaller)	Probado en la VM con Windows 8 (captura 06)
Variables de entorno separadas del código	config/.env.example; .env fuera del repositorio	Cumplido
Dependencias fijadas	requirements.txt y requirements-dev.txt	Cumplido
Esquema de datos versionado	database/migraciones/	Cumplido
Portabilidad como requisito no funcional	RNF-POR-01 a RNF-POR-04	Cumplido
Manual técnico de despliegue	docs/12-despliegue/	Pendiente

Regla de construcción: El ejecutable se genera en una VM con Windows 8.1 o anterior, porque hereda la compatibilidad de la máquina donde se construye.

9. Diseño de la migración

La aplicación no incluye un controlador de Firebird. La migración es un proceso aparte, documentado en docs/09-plan-migracion, porque el Firebird embebido del legado es de 32 bits y no se puede cargar desde un Python de 64 bits.

- Copiar la base del legado y trabajar siempre sobre la copia.
- Respaldo lógico con el gbak 2.0.3 del propio sistema legado y restauración con Firebird 3.0 en la máquina virtual.
- Extracción a CSV y depuración del catálogo, versionada.
- Carga con un importador en Python, probado con archivos de ejemplo.

Reglas para el importador, que salen de defectos encontrados en la primera extracción; no recortar espacios de los códigos sin informar las colisiones que eso provoca, y no redondear importes al extraer.

Implantación en tres etapas: Una semana con ambos sistemas en paralelo, comparación diaria de los cierres, y apagado del sistema antiguo solo si los cierres cuadraron. El respaldo del legado se conserva como reserva, ensayada al menos una vez (HU-MIG-03)

10. Seguridad y datos personales

El detalle está en los requisitos RNF-SEG-01 a RNF-SEG-08. En resumen de diseño:

- Contraseñas solo como hash; con, bcrypt mediante passlib (aprobado el 19-10-2026)
- Autenticación local, para que un corte de internet no deje a la usuaria fuera del sistema.
- Auditoría con usuario, fecha y valores anterior y nuevo, aunque hoy exista un solo usuario.
- Credenciales en variables de entorno y conexión cifrada a Supabase. se rotan si alguna vez se compartieron por un canal no cifrado
- Los datos de clientes no se publican: el repositorio es público por exigencia de la asignatura y las evidencias se revisan antes de subirlas.

11. Temas abiertos de diseño

Tema	Qué falta	Responsable
Contraseñas con bcrypt	Implementar el algoritmo aprobado el 19-10 (RNF-SEG-01)	Fernando Silva · Sprint 5
Rotación de credenciales de Supabase	Rotarlas si se compartieron por WhatsApp u otro canal no cifrado	Fernando Silva
Ensayo con dos cajas	Ejecutar la prueba R-06 antes de ofrecerlo como capacidad (RNF-ESC-02)	Fernando Silva · Sprint 6